

Poglavje 1

Implementacija in analiza Shorovega algoritma za faktorizacijo števil

Blaž Grilj, Ana Poklukar

Povzetek V seminarski nalogi obravnavamo Shorov algoritem kot kvantni pristop k faktorizaciji celih števil in njegove posledice za kriptografske sisteme, kot je RSA. Na kratko predstavimo problem faktorizacije in teoretično ozadje algoritma, zlasti vlogo kvantne Fourierjeve transformacije pri iskanju periode. Nato opišemo implementacijo v Pythonu z uporabo Qiskita ter eksperimentalno preizkusimo algoritem na idealnem simulatorju in IBM kvantnem procesorju. Analiziramo vpliv parametra a , števila ponovitev in velikosti N ter rezultate primerjamo z brute-force pristopom in Pollardovim ρ -algoritmom. Pokazali smo, da kvantna implementacija kljub teoretični učinkovitosti uspešno zazna periodičnost za večja števila, vendar njeno delovanje v praksi omejuje šum in majhno število stabilnih kubitov.

1.1 Uvod

Kvantno računalništvo predstavlja eno najperspektivnejših smeri nadaljnjega razvoja procesnih tehnologij, saj obljublja reševanje problemov, ki so za klasične računalnike izračunljivo zelo zahtevni ali praktično nerešljivi. Med najpomembnejšimi kvantnimi algoritmi izstopa Shorov algoritem, ki omogoča eksponentno hitrejšo faktorizacijo velikih kompozitnih števil. Ker varnost sodobnih kriptografskih sistemov, kot je RSA [10], neposredno temelji na težavnosti faktorizacije, ima Shorov algoritem pomembne posledice za področje informacijske varnosti.

V tej seminarski nalogi raziskujemo Shorov algoritem z vidika teoretičnih temeljev, implementacije in praktične uporabnosti v trenutnem stanju kvantne tehnologije. Najprej predstavimo problem faktorizacije celih števil in teoretično ozadje Shorovega algoritma, nato opišemo našo implementacijo v okolju Qiskit [11] ter zasnovo eksperimentalnih preizkusov. V nadaljevanju analiziramo rezultate, pridobljene na idealnem simulatorju in na pravem IBM kvantnem procesorju [12], ter jih primerjamo z izbranimi klasičnimi metodami faktorizacije. Na koncu povzamemo ključne ugotovitve in izpostavimo omejitve trenutnega pristopa.

1.2 Metode

To poglavje podrobno predstavlja matematične osnove, uporabljene algoritme ter praktične postopke, ki so bili uporabljeni pri analizi, implementaciji in eksperimentalnem preizkušanju Shorovega algoritma.

1.2.1 Problem faktorizacije celih števil

Problem faktorizacije celih števil obravnava razcep sestavljenega števila N na njegova praštevilska faktorja. Čeprav je naloga na prvi pogled preprosta, postane izračunsko zahtevna, ko je N zelo veliko (npr. 1024-bitno ali 2048-bitno število). Klasični računalniki ne poznajo polinomske hitre metode za faktorizacijo splošnih celih števil, zato je ta problem temelj številnih kriptografskih sistemov, med katerimi je najpomembnejši RSA.

Velika sestavljena števila se v kriptografiji uporabljajo kot del javnega ključa. V RSA-ju je javni ključ sestavljen iz para (N, e) , kjer je $N = pq$ produkt dveh velikih praštevil, medtem ko sta p in q skrita. Varnost sistema temelji na tem, da je iz javnega ključa enostavno izvajati šifriranje, medtem ko je dešifriranje možno le, če poznamo zasebni ključ, ki je odvisen od faktorjev p in q . Ker je faktorizacija velikega N izjemno težavna, lahko sistemi varno uporabljajo tudi 2048-bitne ali 4096-bitne ključe, pri čemer bi klasični računalniki za razcep takih števil potrebovali veliko časa.

V praksi se danes uporabljajo različni klasični algoritmi za faktorizacijo, kot so preizkušanje deliteljev (brute-force), Pollardov ρ -algoritem [13], kvadratna sita [14] in GNFS [15]. Ti algoritmi imajo različne prednosti, vendar vsi za dovolj velike vrednosti N postanejo časovno neučinkoviti. Shorov algoritem predstavlja radikalno drugačen, kvantni pristop k faktorizaciji, saj problem reducira na iskanje periode in ga reši v polinomskem času. Zaradi njegove učinkovitosti bi dovolj velik kvantni računalnik, zmožen poganjati Shorov algoritem, ogrozil večino danes uporabljenih kriptografskih sistemov, kar je glavni razlog za intenzivno raziskovanje in razumevanje tega algoritma.

1.2.2 Teoretično ozadje Shorovega algoritma

Shorov algoritem, ki ga je leta 1994 predstavil Peter W. Shor [16], predstavlja enega najpomembnejših mejnikov v razvoju kvantnega računalništva, saj omogoča faktorizacijo velikih sestavljenih celih števil v polinomskem času. Ker varnost kriptografskih shem, kot je RSA, temelji na težavnosti faktorizacije, ima Shorov algoritem izjemen teoretični in praktični pomen.

Shorov algoritem temelji na preoblikovanju problema faktorizacije v problem iskanja periodičnosti. Naj bo N sestavljeno število, ki ga želimo razstaviti na prafak-

torje. Izberemo naključno število a , za katerega velja $1 < a < N$ ter $\gcd(a, N) = 1$. Definiramo funkcijo

$$f(x) = a^x \bmod N,$$

ki je periodična z najmanjšo pozitivno periodo r , tako da velja $a^r \equiv 1 \pmod{N}$. Če je r soda, lahko iz periodičnosti izpeljemo prafaktorja števila N z izračunom

$$p_1 = \gcd(a^{r/2} - 1, N), \quad p_2 = \gcd(a^{r/2} + 1, N),$$

pri čemer bo z visoko verjetnostjo vsaj eden od teh izrazov dal netrivialen prafaktor. Če pa je r liho število, metoda ne deluje, saj $r/2$ ni celo število, zato je treba izbrati novo vrednost a in postopek ponoviti.

Ključni prispevek Shorovega algoritma je uporaba kvantne Fourierjeve transformacije (*ang.* Quantum Fourier Transform ali QFT), ki omogoča iskanje periode r v polinomskem času. Kvantna Fourierjeva transformacija v prostoru dimenzije 2^n , kjer je n število uporabljenih kvantnih bitov, je definirana kot linearna transformacija

$$\text{QFT}_{2^n} : |x\rangle \mapsto \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i x k / 2^n} |k\rangle.$$

QFT preslika stanje $|x\rangle$ v valovno obliko, kjer superpozicija vseh baznih stanj $|k\rangle$ vsebuje faze, odvisne od produkta xk . Ti fazni premiki ustvarijo interferenčni vzorec, iz katerega lahko z veliko verjetnostjo razberemo periodo funkcije $f(x)$. Ključni razlog, da to deluje, je dejstvo, da se periodičnost funkcije $f(x)$ odraža v periodičnih amplitudah prvega registra, na katere QFT zelo učinkovito reagira.

Ko enkrat z uporabo kvantne Fourierjeve transformacije dobimo oceno periode r , lahko preostanek postopka izvedemo klasično. Če je r soda in velja $a^{r/2} \not\equiv -1 \pmod{N}$, izračunamo vrednosti $\gcd(a^{r/2} - 1, N)$ in $\gcd(a^{r/2} + 1, N)$, ki z visoko verjetnostjo vrmeta netrivialna prafaktorja števila N . Če dobimo liho periodo ali neuporaben rezultat, izberemo nov a in postopek ponovimo, dokler ne najdemo primerne periode.

Časovna zahtevnost celotnega algoritma izhaja iz kvantnega dela, ki vključuje modularno potenciranje in kvantno Fourierjevo transformacijo. Ti operaciji zahtevata največ $\mathcal{O}(n^3)$ časa, kjer je $n = \log_2 N$. To pomeni, da Shorov algoritem faktorizira števila v polinomskem času, kar je izjemno izboljšanje v primerjavi s klasičnimi metodami, ki imajo lahko eksponentno zahtevnost.

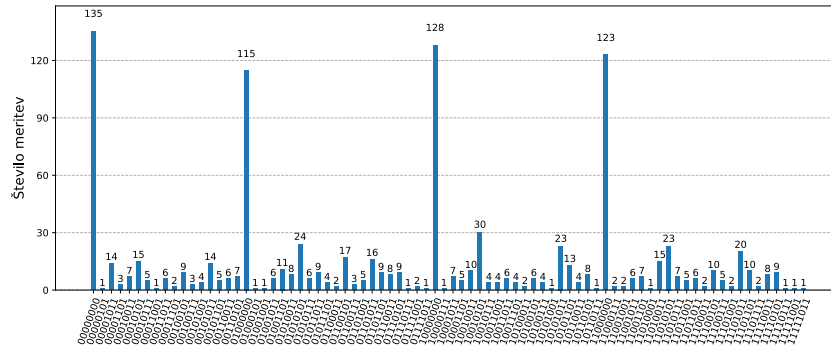
1.2.3 Opje implementacije

V tem podpoglavju je predstavljena implementacija Shorovega algoritma za faktorizacijo števila N .

Program je napisan v Pythonu in uporablja ogrodje Qiskit, ki omogoča gradnjo in izvajanje kvantnih vezij tako na simulatorjih kot tudi na dejanskih IBM kvantnih procesorjih. Implementacija združuje kvantni del in klasično obdelavo rezultatov.

Koraki implementacije so naslednji: najprej naključno izberemo število a , ki je med 2 in $N - 1$. Takoj preverimo največji skupni delitelj d . Če je $d \neq 1$, smo že našli netrivialni faktor števila N in ga vrnemo kot rešitev. Če $d = 1$, nadaljujemo z gradnjo kvantnega vezja za ocenjevanje faze (*ang.* Quantum Phase Estimation ali QPE) z modularnim množenjem izbranega števila a .

Vezje nato izvedemo na simulatorju ali kvantnem računalniku in pridobimo histogram izmerjenih rezultatov. V simulaciji je ta histogram dostopen v celoti, saj simulator izvede veliko neodvisnih meritev in nam vrne celotno porazdelitev verjetnosti. V izvirnem Shorovem algoritmu pa QPE vrne le eno samo meritev, saj je kvantno vezje mogoče izvesti le enkrat na dejanski napravi, rezultat pa je posamezna bitna vrednost. Dostop do celotne porazdelitve (histograma) je zato umetna prednost simulacije in omogoča podrobnejšo analizo statistike meritev. Primer porazdelitve meritev za naključno izbran a in $N = 15$ pri $\text{shots} = 1024$ je prikazan na sliki 1.1, kjer so jasno vidni vrhovi, ki ustrezajo približkom periode.



Slika 1.1: Histogram izmerjenih vrednosti iz simulacije QPE.

V naši implementaciji zato iz histograma izberemo meritev z največjo verjetnostjo, ki predstavlja najbolj stabilen približek izmerjene faze. Iz te meritve pretvorimo bitni niz v decimalno vrednost in na podlagi nje s pomočjo nadaljevalnih ulomkov izračunamo kandidat za periodo zaporedja. Če je dobljena perioda smiselna, izračunamo dva možna delitelja števila N na osnovi polovice periode. Če sta oba delitelja netrivialna, smo uspešno razcepili število N . Če kandidat za periodo ne vodi do uporabnih deliteljev, postopek označimo kot neuspešen. Psevdokoda korakov implementacije je prikazana v algoritmu 1.

1.2.3.1 Konstrukcija kvantnega vezja

V nadaljevanju na kratko, a nekoliko podrobneje razložimo kvantni del implementacije Shorovega algoritma.

Algorithm 1 Shorov algoritem

```

izberi naključno število  $2 \leq a < N$ ,  $d = \gcd(a, N)$ 
if  $d \neq 1$  then
    return  $d$ 
end if
zgradi kvantno vezje za fazno ocenjevanje z modularnim množenjem števila  $a$ 
zaženi vezje in pridobi histogram meritev
izberi meritev z največjo verjetnostjo
pretvori meritev v decimalno vrednost
izračunaj periodo
if je dobljena perioda ustrezna then
    preveri delitelja, dobljena iz polovice periode
    if sta delitelja netrivialna then
        return faktorja števila  $N$ 
    end if
end if
return neuspeh

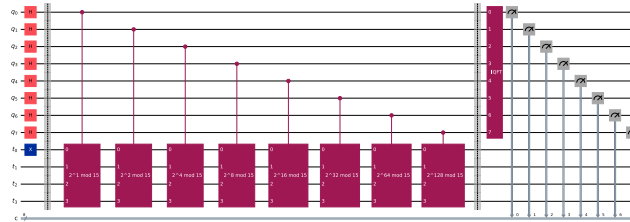
```

Konstrukcija vezja temelji na algoritmu za ocenjevanje faze QPE, ki predstavlja osrednji kvantni del Shorovega algoritma. Postopek se začne z določitvijo velikosti registrov glede na bitno dolžino števila N , ki ga želimo faktorizirati. V ta namen vzpostavimo dva kvantna registra: merilni (oziroma števeni) register, ki obsega dvakratno število kubitov, in ciljni register za shranjevanje stanja. Za beleženje končnih rezultatov dodamo še klasični register ustrezne velikosti.

V začetni fazi ciljni register inicializiramo v stanje $|1\rangle$, kar predstavlja lastno stanje za operacijo modularnega množenja. Hkrati vse kubite v merilnem registru s Hadamardovimi vrati postavimo v superpozicijo, s čimer omogočimo interferenco in zajem fazne informacije. Jedro vezja tvori zaporedje kontroliranih operacij modularnega množenja. Za vsak kubit v merilnem registru izvedemo specifična kontrolirana vrata, ki na ciljnem registru aplicirajo operacijo $a^{2^k} \pmod{N}$. Ker je ta logika vezana na potence števila 2, s tem učinkovito zgradimo operator modularnega potenciranja.

Proces zaključimo z inverzno kvantno Fourierjevo transformacijo na merilnem registru. Ta korak prevede skrito fazno informacijo v verjetnostno porazdelitev stanj, ki jo z meritvijo preslikamo v klasični register.

Na sliki [1.2](#) lahko vidimo vizualizacijo kvantnega vezja za primer $N = 15$. Kubiti so najprej poslani skozi Hadamardova vrata, kar jih postavi v superpozicijo vseh možnih vrednosti x . Veliki kontrolirani bloki z oznakami $13^{(2^k)} \pmod{15}$ predstavljajo korake modularnega potenciranja, ki izračunajo $13^x \pmod{15}$. Spodnji kubiti služijo kot register, v katerega se zapisuje rezultat teh izračunov. Blok inverzne kvantne Fourierjeve transformacije (*ang.* Inverse Quantum Fourier Transform ali IQFT) na desni pretvori stanje zgornjega registra tako, da se v njem razkrije perioda funkcije. Merilniki na koncu izmerijo zgornje kubite in tako podajo vrednost, iz katere se lahko izračuna perioda.



Slika 1.2: Kvantno vezje, zgrajeno za fazno ocenjevanje v Shorovem algoritmu.

1.3 Rezultati

V tem poglavju predstavimo rezultate eksperimentalnih preizkusov Shorovega algoritma, izvedenih tako na lokalnem simulatorju kot na pravem IBM kvantnem procesorju. Vsi lokalni izračuni so bili izvedeni na osebnem računalniku z Intel Core i7–9700 procesorjem. Rezultati vključujejo analizo vpliva izbire števila a , vpliv števila ponovitev, primerjavo med simulatorjem in fizično napravo ter primerjavo z izbranimi klasičnimi faktorizacijskimi metodami.

1.3.1 Vpliv izbire števila a

Najprej smo se lotili preverjanja, kako izbira parametra a vpliva na uspešnost faktorizacije v Shorovem algoritmu. Parameter a je osrednji del postopka, saj določa funkcijo $f(x) = a^x \bmod N$, katere periodo poskuša algoritem najti. V izvorni različici algoritma je a izbran naključno, v našem poskusu pa smo ga za potrebe analize nastavili na fiksno vrednost in sistematično preizkusili vse dovoljene vrednosti a , za katere velja $\gcd(a, 15) = 1$. Število N smo postavili na 15, parameter `shots` nastavili na 1024, nato pa smo za vsak izbran a izvedli 100 ponovitev algoritma in izračunali delež uspešno izvedenih faktorizacij. V tabeli 1.1 so prikazane povprečne uspešnosti za posamezne vrednosti a .

Iz dobljenih rezultatov lahko vidimo, da se delež uspešnih faktorizacij med različnimi vrednostmi a nekoliko razlikuje, vendar so te razlike razmeroma majhne. To je pričakovano, saj ima večina dovoljenih vrednosti a za $N = 15$ podobno obnašanje pri modularnem potenciranju, kar vodi do enakomernih meritev periode. Podobne rezultate bi dobili tudi pri večini drugih izbranih vrednosti N , kjer bi bila uspešnost za različne vrednosti a precej enakomerna. Slabša uspešnost bi se pojavila predvsem v primerih, ko ima izbrani a neustrezno periodo (npr. liho periodo ali zelo majhno periodo), zaradi česar algoritem ne bi mogel pravilno izračunati faktorizacije.

a	Uspešnost [%]
2	77
4	45
7	82
8	77
11	50
13	73
14	76

Tabela 1.1: Vpliv izbire števila a na uspešnost faktorizacije za fiksno $N = 15$.

1.3.2 Vpliv števila ponovitev

Nato smo raziskovali, kako število ponovitev kvantnega vezja (parameter `shots`) vpliva na uspešnost faktorizacije. Parameter `shots` določa, kolikokrat zapored se izvede isto kvantno vezje in izvede meritev, kar neposredno vpliva na kakovost statistične porazdelitve izidov. Večje število ponovitev praviloma pomeni natančnejši približek dejanske verjetnostne porazdelitve po QFT. Tudi v tem poskusu smo uporabili fiksno vrednost $N = 15$, parameter a pa je bil izbran naključno. Za vsako izbrano število `shots` smo algoritem izvedli 100-krat in izračunali povprečen delež uspešno izvedenih faktorizacij, kot je prikazano v tabeli [1.2](#).

Št. ponovitev (<code>shots</code>)	Uspešnost [%]
1	65
128	79
512	73
1024	80
4096	83

Tabela 1.2: Vpliv števila ponovitev (`shots`) na uspešnost faktorizacije za fiksno vrednost $N = 15$.

Rezultati v tabeli kažejo, da se uspešnost faktorizacije na splošno izboljšuje z naraščajočim številom ponovitev. Razlog za to je, da večje število `shots` ustvari natančnejšo statistično porazdelitev meritev, iz katere lahko algoritem bolj zanesljivo razbere približek periode. Pri zelo majhnem številu meritev (npr. `shots` = 1) je rezultat praktično naključen, saj posamezna meritev ne odraža dejanskih amplitud po kvantni Fourierjevi transformaciji. Pri največjih vrednostih `shots` se uspešnost stabilizira, saj nadaljnje povečanje ne prinese bistvenega dodatnega statističnega izboljšanja za tako majhno število N .

1.3.3 Primerjava idealnega simulatorja in pravega kvantnega procesorja

V tem delu smo opravili primerjavo med idealnim kvantnim simulatorjem, v katerem smo izvajali našo implementacijo Shorovega algoritma, ter pravim kvantnim procesorjem IBM.

Idealni simulator v okolju Qiskit predstavlja model kvantnega računalnika brez šuma, napak pri meritvah ali netočnosti kvantnih vrat, medtem ko IBM-jev kvantni procesor predstavlja resnično strojno napravo, kjer vezje teče na kvantnih bitih, ki so občutljivi na šum in generirajo meritve z napakami. Za primerjavo smo izbrali več različnih vrednosti N , parameter `shots` nastavili na 1024 in za vsak primer izmerili čas izvajanja ter uspešnost faktorizacije. Lokalni simulator smo pognali 100-krat in izračunali povprečje uspešnosti, medtem ko smo na IBM procesorju izvedli le po 10 ponovitev za vsako vrednost N , saj je dostop do pravega kvantnega računalnika omejen in drag.

N	Čas [s]	Uspešnost [%]
15	0.068	80
21	0.080	63
33	0.085	53
143	0.804	32
314191	/	/

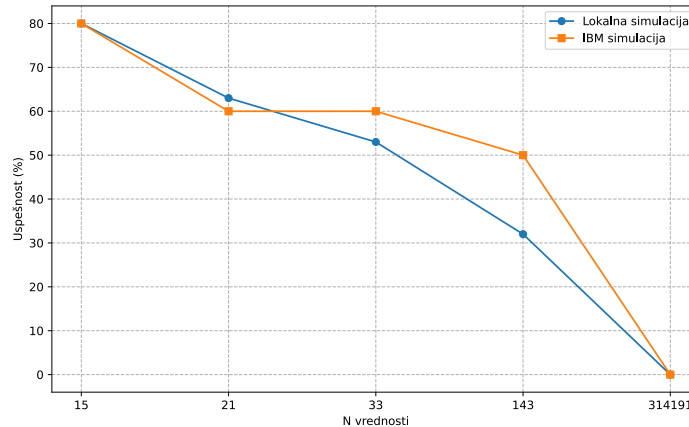
Tabela 1.3: Rezultati faktorizacije za različne vrednosti N , izvedeni na lokalnem idealnem simulatorju (100 ponovitev).

N	Čas [s]	Uspešnost [%]
15	2	80
21	2	60
33	2	60
143	2	50
314191	3	0

Tabela 1.4: Rezultati faktorizacije za različne vrednosti N , izvedeni na IBM kvantnem procesorju (10 ponovitev).

Rezultati v tabelah 1.3 in 1.4 kažejo, da čas izvajanja na lokalnem simulatorju z velikostjo N narašča, a je še vseeno bistveno krajši kot na IBM kvantnem procesorju. To je pričakovano, saj simulator izvede izračun klasično in brez dodatnih režijskih časov, ki jih na IBM-ju predstavljajo čakalne vrste, prenosi podatkov, inicializacija vezja ter fizične omejitve naprave. Uspešnost faktorizacije je razmeroma podobna pri obeh pristopih, IBM procesor nekoliko bolje deluje pri večjih vrednostih N , kar lahko opazimo tudi na sliki 1.3. Na največjem testiranem številu lokalni simulator

sploh ni mogel izvesti algoritma, saj smo bili omejeni na največ 30 kvantnih bitov, medtem ko je IBM računalnik kljub šumu uspel vezje izvesti in neuspeh vrniti v nekaj sekundah. Upoštevati moramo tudi, da je bila statistika IBM-jevih rezultatov manj zanesljiva, ker smo lahko vsako vrednost N pognali le desetkrat.



Slika 1.3: Primerjava uspešnosti faktorizacije med lokalnim simulatorjem in IBM kvantnim procesorjem za različne vrednosti N .

1.3.4 Primerjava s klasičnimi metodami faktorizacije

Na koncu smo primerjali in analizirali klasične pristope za faktorizacijo s kvantno implementacijo Shorovega algoritma. Primerjali smo tri metode: naivni brute-force pristop, ki preizkusi vse možne delitelje do $\sqrt{N}$, Pollardov ρ -algoritem, ki predstavlja eno izmed najhitrejših klasičnih naključnostnih metod za faktorizacijo srednje velikih števil, ter našo implementacijo Shorovega algoritma, izvedeno na IBM kvantnem procesorju. Za vsako metodo smo merili čas izvajanja in izbrali tri vrednosti N , ki pokrivajo majhno, srednjo in večjo velikost problema. Parameter `shots` nastavili na 1024, izbira a je bila naključna. Izmerjeni časi predstavljajo povprečen čas, potreben za uspešno faktorizacijo.

Rezultati v tabeli [1.5](#) kažejo, da je pri majhnih vrednostih N kvantna implementacija daleč najpočasnejša, kar je pričakovano, saj IBM kvantni procesorji vključujejo dodatno zakasnitev zaradi inicializacije sistema in samega izvajanja kvantnega vezja. Klasični metodi v tem primeru porabita zanemarljivo malo časa. Tudi pri

N	Brute-force čas [s]	Pollard ρ čas [s]	Kvantni čas [s]	Št. kubitov
15	0	0	2	12
314191	0.001	0	2	57
384210388873	/	0.002	10	117

Tabela 1.5: Primerjava časa izvajanja brute-force metode, Pollardovega ρ algoritma in kvantne implementacije Shorovega algoritma za različne vrednosti N .

srednjih vrednostih N se razmerje bistveno ne spremeni. Pri največjem izbranem številu brute-force pristop odpove zaradi prevelikega iskalnega prostora, Pollardov ρ pa še vedno uspe hitro najti faktorje. Kvantna implementacija v tem primeru potrebuje bistveno več časa, ker število kubitov hitro narašča z velikostjo N , globina kvantnega vezja postane zelo velika, posledično pa se povečajo tudi napake, zato je potrebno več ponovitev. Za občutek zahtevnosti naloge smo v tabelo vključili tudi število uporabljenih kubitov, ki narašča z velikostjo N . Za dodatno orientacijo naj omenimo, da ima uporabljen IBM-jev procesor največ 156 kubitov, medtem ko njihov trenutno največji procesor Condor vsebuje 1121 kubitov [17], ampak ta ni na voljo širši javnosti.

1.4 Zaključek

V seminarski nalogi smo preučili Shorov algoritem z vidika teoretičnih temeljev, praktične izvedbe in primerjave z izbranimi klasičnimi metodami faktorizacije. Implementacija algoritma v okolju Qiskit nam je omogočila podrobno analizo vpliva parametra a , števila ponovitev ter obnašanja algoritma na idealnem simulatorju in pravem IBM kvantnem procesorju. Rezultati so potrdili znano dejstvo, da idealni simulator dosega visoko stabilnost in zanesljivost, a nas omeji s številom kubitov, medtem ko je uspešnost na pravih kvantnih napravah omejena predvsem zaradi šuma in napak zaradi globine vezja. Primerjava s klasičnimi metodami je pokazala, da kvantni pristop pri trenutni tehnologiji še ni konkurenčen nobeni zmogljeni klasični metodi, saj zahteva bistveno več časa in je manj stabilen.

Kljub temu pa Shorov algoritem ostaja izjemno pomemben dokaz koncepta, ki kaže na velik potencial kvantnega računalništva. Njegova teoretična časovna zahtevnost je namreč polinomska, kar pomeni, da bi lahko ob zadostnem številu stabilnih kubitov in znatno manjšem šumu kvantni računalniki v prihodnosti dejansko prehiteli tudi najučinkovitejše klasične faktorizacijske algoritme. Pričakovati je, da se bo z napredkom kvantnih naprav ter boljšimi tehnikami zmanjševanja šuma njegova praktična uporabnost postopoma povečevala. Naše delo tako predstavlja majhen prispevek k razumevanju trenutnega stanja tehnologije in smernicam za nadaljnji razvoj kvantnih algoritmov.

1.5 Doprinosi avtorjev

Avtor Blaž Grilj je implementiral Shorov algoritem in vse uporabljene klasične metode za primerjavo, pripravil podpoglavje o implementaciji in vizualizacije, izvedel vse eksperimentalne poskuse in naredil prosojnice za predstavitev. Avtorica Ana Poklukar je zasnovala strukturo seminarske naloge, napisala uvod, teoretični del poglavja Metode (razen implementacijskega dela), pripravila analize rezultatov ter oblikovala zaključek naloge.